

Информационные системы в химической технологии
Практическая работа № 12

**Изучение асинхронного взаимодействия приложений.
Настройка менеджера очередей**

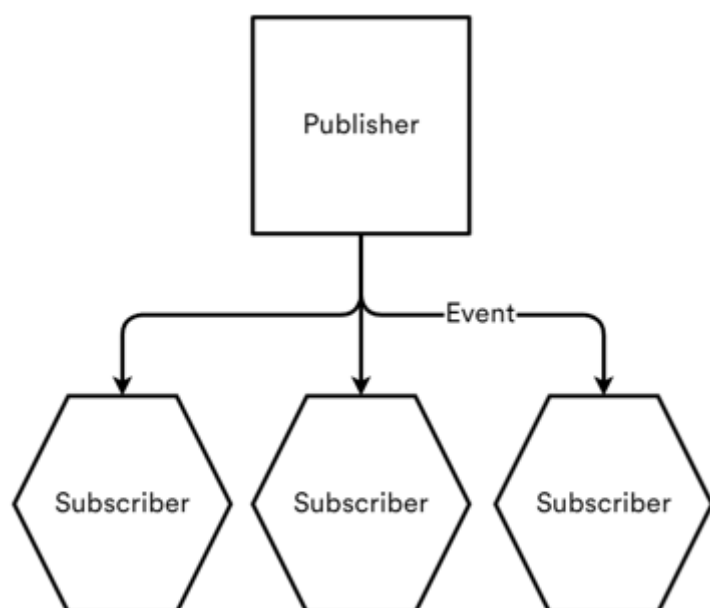
Цель работы

Ознакомиться с основами асинхронного обмена между приложениями.

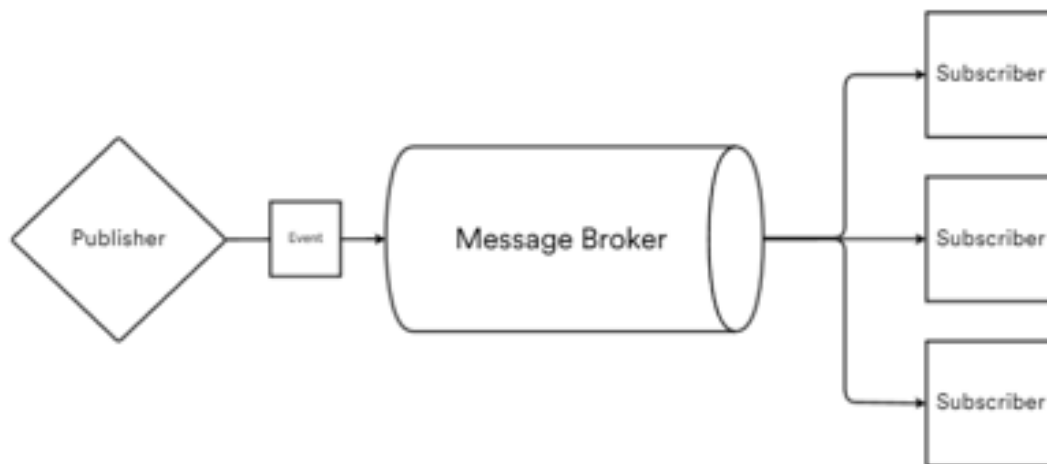
Ознакомиться с основными командами для конфигурации и настройки менеджера очередей MQ.

Брокеры сообщений — это приложения-посредники, используемые в различных видах архитектуры программного обеспечения, например, в микросервисах. Брокеры пересылают информацию в виде сообщений от сервиса к сервису.

С брокерами сообщений тесно связан паттерн «publisher-subscriber», суть которого заключается в передаче некоторых событий от так называемых «издателей» (Producers) к потребителям (Consumers).



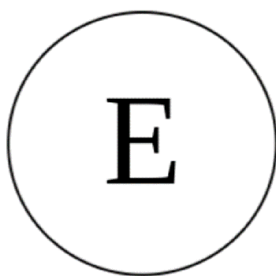
Чаще всего в реализации этого паттерна участвует промежуточное звено — тот самый брокер сообщений.



Три главных сущности брокера сообщений: Exchange, Queue, Binding.

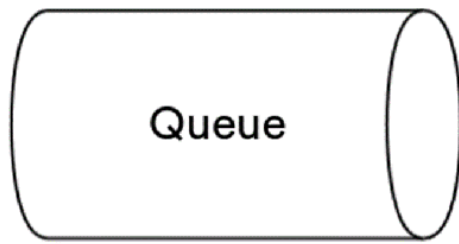
Exchange

Сюда попадают сообщения-события. Exchange решает, в какую очередь какие события попадут. Это происходит на основе связей — байндингов, о которых будет сказано далее. На схеме Exchange таким способом:



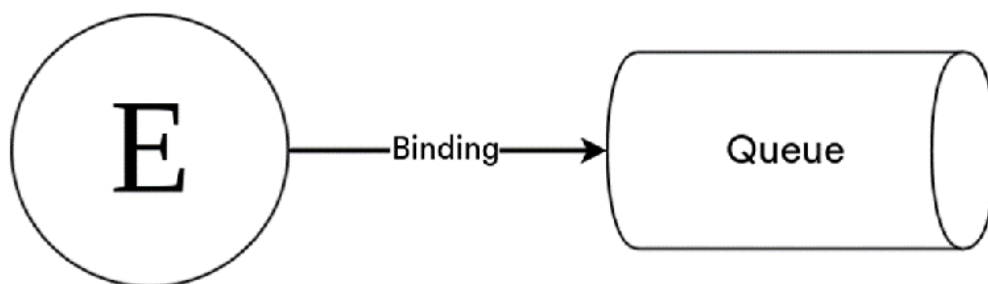
Queue

Или просто очередь. Структура данных, получение которых построено по принципу «первый вошёл — первый вышел», или FIFO. Данные хранятся на диске или в ОЗУ и представляют непосредственно сообщения. Очередь отдает копии этих данных потребителям (consumers). Схематично изобразим очередь:



Binding

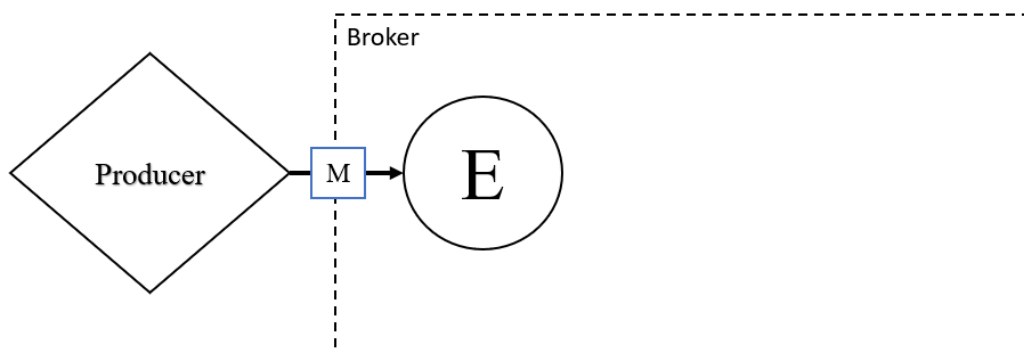
Привязка. Это набор правил, которые указывают Exchange, в какую из очередей должны попадать сообщения. Между Exchange и Queue может быть несколько привязок с разными параметрами.



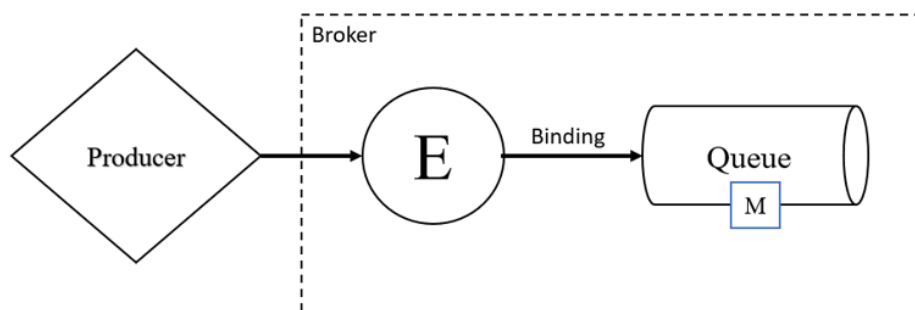
Принцип работы

Producer — это сервис, который генерирует сообщения и посылает их брокеру (другими словами, отправитель). Consumer — это другой сервис, который при получении сообщения начинает его обработку (получатель).

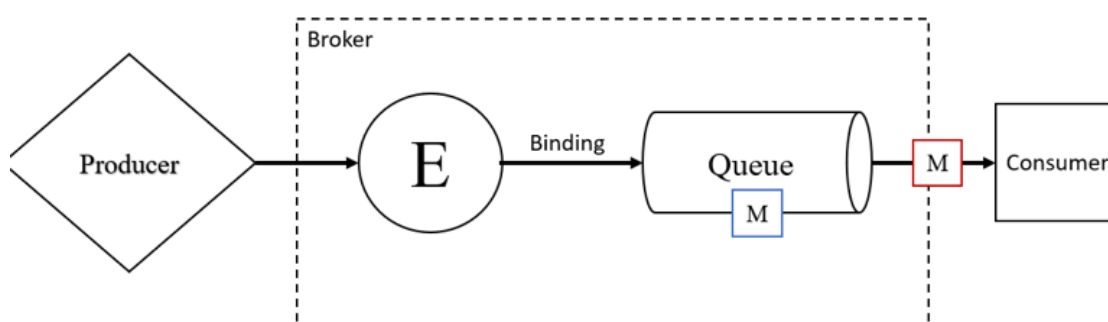
1. Producer отправляет сообщение, и оно попадает в Exchange.



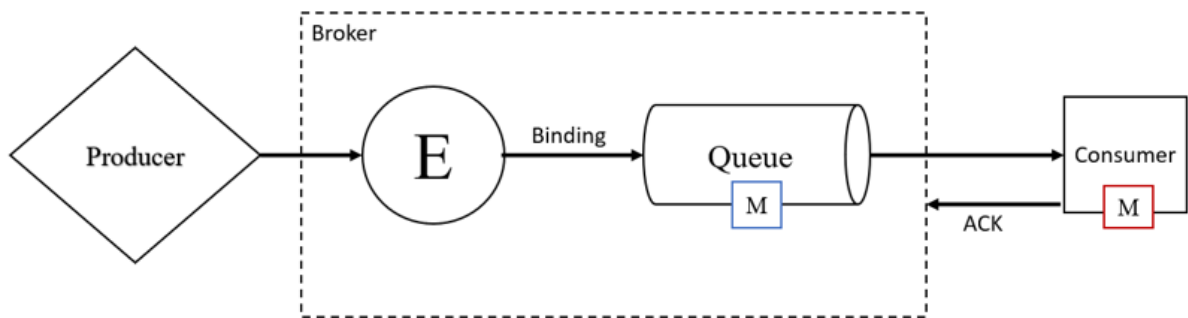
2. Очередь хранит это сообщение — например, на диске (зависит от реализации брокера).



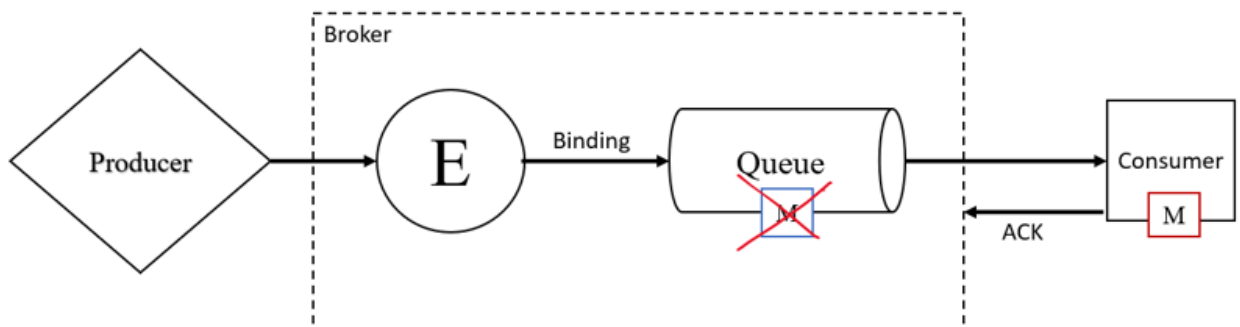
3. Когда consumer готов принять сообщение (закончил обработку предыдущего или просто приложение-обработчик событий запустилось) — сервер посылает в него скопированные данные из очереди.



4. Consumer получает сообщение, затем каким-то образом обрабатывает его и передаёт брокеру подтверждение успешного принятия сообщения (ACK).



5. После получения брокером подтверждения сообщение удаляется.



RabbitMQ

Это популярный open-source брокер сообщений. Он базируется на основе протокола AMQP (Advanced Message Queuing Protocol) — это открытый протокол для передачи сообщений-событий через специальный брокер. Сам протокол имеет очень много возможностей, и помимо RabbitMQ его реализует, например Apache Qpid.

Сам «кролик» разработан на языке программирования Erlang, а его основные преимущества - пропускная способность и максимальная гибкость маршрутизации. Для «кролика» существует множество вариантов настройки правил того, какие сообщения и куда будут попадать на обработку.

Подключаемся к Рэббиту мы по протоколу TCP: клиент инициализирует подключение по адресу хоста и сохраняет соединение до тех пор, пока ему

необходимо взаимодействовать с RabbitMQ. Он также поддерживает аутентификацию, например, по логину и паролю

Установка на Astra Linux

RabbitMQ состоит из двух частей: сервер и WebUI (административный интерфейс). В UI можно смотреть, что, происходит в брокере: работоспособны ли узлы кластера, сколько сообщений в процессе обработки и т.д.

1. Установите пакет для приложения rabbitmq

```
sudo apt install -y rabbitmq-server
```

где:

-y - ключ для пропуска подтверждения установки.

2. После успешного завершения установки необходимо приступить к настройке RabbitMQ:

- a) Перейдите в интерфейс командной строки «Терминал Fly» и убедитесь в наличии каталога `/etc/rabbitmq`:

```
ls /etc/ | grep rabbitmq
```

- b) Если каталог отсутствует, необходимо его создать:

```
sudo mkdir -p /etc/rabbitmq
```

где:

- p - ключ для автоматического создания директорий, указанных внутри пути.

- c) перейдите в каталог `/etc/rabbitmq`:

```
cd /etc/rabbitmq
```

- d) Создайте пустые файлы `rabbitmq.conf` (основной файл конфигурации RabbitMQ) и `definitions.json` (файл определения данных для подключения):

```
sudo touch rabbitmq.conf
sudo touch definitions.json
```

- e) Поменяйте владельца (rabbitmq) и идентификатор группы (rabbitmq) для созданных файлов:

```
sudo chown rabbitmq:rabbitmq definitions.json
rabbitmq.conf
```

- f) Отредактируйте файл rabbitmq.conf, приведя его к виду:

```
# ===== Management section
=====
## Preload schema definitions from the following JSON file.
## Related doc guide: https://rabbitmq.com/management.html#load-
definitions.
##
# management.load_definitions = /path/to/exported/definitions.json
management.load_definitions = /etc/rabbitmq/definitions.json
tcp_listen_options.keepalive = true
```

- g) Отредактируйте файл definitions.json, задав пользователей termidesk и admin, приведя его к виду:

В файле /etc/rabbitmq/definitions.json пароли указываются в виде преобразованного значения, которое можно получить через исполняемый файл rabbitmq_password2hash.sh. Процесс преобразования пароля будет приведен далее.

```
{
  "rabbit_version": "3.7.8",
  "users": [
    {
      "name": "termidesk",
      "password_hash":
"pnXiDJtUdk7ZceL9iOqx44PeDgRa+Xl+eIq+7wf/PTONLb1h",
      "hashing_algorithm":
"rabbit_password_hashing_sha256",
      "tags": ""
    },
    {
      "name": "admin",
      "password_hash":
"FXQ9WFNSrsGwRki9BT2dCITnsDwYu2lsy7BEN7+UncsPzCDZ",
      "hashing_algorithm":
"rabbit_password_hashing_sha256",
      "tags": "administrator"
    }
  ],
  "vhosts": [
    {
      "name": "/"
    },
    {
      "name": "termidesk"
    }
  ]
}
```

```

],
"permissions": [
  {
    "user": "termidesk",
    "vhost": "termidesk",
    "configure": ".*",
    "write": ".*",
    "read": ".*"
  },
  {
    "user": "admin",
    "vhost": "termidesk",
    "configure": ".*",
    "write": ".*",
    "read": ".*"
  }
],
"topic_permissions": [
  {
    "user": "termidesk",
    "vhost": "termidesk",
    "exchange": "",
    "write": ".*",
    "read": ".*"
  }
],
"parameters": [],
"global_parameters": [
  {
    "name": "cluster_name",
    "value": "rabbit@rabbitmq"
  }
],
"policies": [],
"queues": [],
"exchanges": [],
"bindings": []
}

```

В примере используются следующие пары логин-пароль: admin:admin, termidesk:ksedimret.

Имя пользователя и пароль должны задаваться в соответствии с внутренними стандартами организации по применению парольной защиты. Для задания пароля разрешено использовать только латинские буквы A-Z, a-z, цифры 0-9 и символы !\$%^&_-=+~`.,()*{}[]\.

h) Выполните команду инициализации RabbitMQ:

```
sudo rabbitmq-plugins enable rabbitmq_management
```

i) Перезапустите службу rabbitmq-server:

```
sudo systemctl restart rabbitmq-server
```

j) Выполните настройку поддержки TCP для ОС путем внесения следующих изменений в файл /etc/sysctl.conf:

```
net.ipv4.tcp_keepalive_time = 30
net.ipv4.tcp_keepalive_intvl = 10
net.ipv4.tcp_keepalive_probes = 3
net.ipv4.tcp_retries2 = 5
```

к) Что бы изменения вступили в силу выполните следующую команду:

```
sudo sysctl --system
```

Опциональная часть

Чтобы получить преобразованное значение пароля, нужно воспользоваться исполняемым файлом `rabbitmq_password2hash.sh`, расположенным в подключенном репозитории Termidesk (см. [Получение пакетов установки в ОС Astra Linux Special Edition](#)) по пути `/var/repos/Addons/Scripts/`. Для этого:

- перейдите в каталог `/var/repos/Addons/Scripts/`:

```
cd /var/repos/Addons/Scripts/
```

- задайте файлу `rabbitmq_password2hash.sh` флаг для запуска:

```
sudo chmod +x rabbitmq_password2hash.sh
```

где:

`+x` - ключ установки разрешения на запуск файла для всех пользователей;

- Запустите на исполнение файл:

```
sudo ./rabbitmq_password2hash.sh
```

При выполнении исполняемого файла будет предложено ввести пароль и повторить его. Преобразованное значение введенного пароля отображается в интерфейсе командной строки.

Полученное значение нужно внести в файл `/etc/rabbitmq/definitions.json` (секция `password_hash`).

Если конфигурационный файл `/etc/rabbitmq/definitions.json` уже существует, но требуется изменить в нем значение преобразованного пароля, для этого необходимо выполнить следующие действия:

- выполнить преобразование и добавление пароля в файл `/etc/rabbitmq/definitions.json`:

```
sudo ./rabbitmq_password2hash.sh -w -u <user> -p <пароль>
```

где:

`-w` - ключ для сохранения изменений в файл `/etc/rabbitmq/definitions.json`;

`-u` - пользователь RabbitMQ. В стандартной установке можно указывать `admin`;

-p - пароль;

- перезапустить службу `rabbitmq-server`:

```
sudo systemctl restart rabbitmq-server
```